Status	Finished
Started	Wednesday, 28 October 2026, 10:45 PM
Completed	Wednesday, 28 October 2026, 10:57 PM
Duration	11 mins 57 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

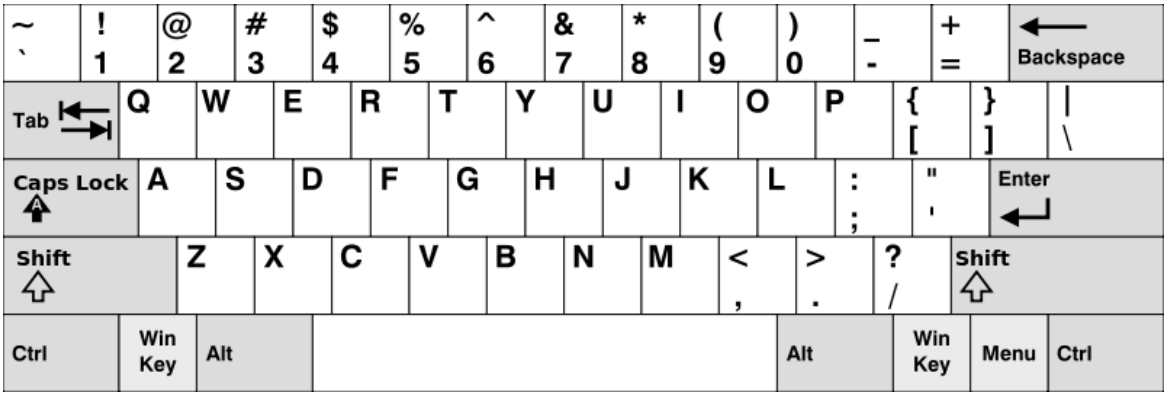
Incorrect

Mark 0.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below.*

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all lines from standard input
    raw_input = sys.stdin.read().splitlines()
    if not raw_input:
        return

    words = []

    # Process lines: handle optional integer N on first line or direct words
    for i, line in enumerate(raw_input):
```

```
        cleaned = line.strip()
        if not cleaned:
            continue

        # If first non-empty line is purely a number, treat it as length count N
        if i == 0 and cleaned.isdigit():
            continue

        # Otherwise, add valid words
        words.append(cleaned)

    # Keyboard rows as character sets
    row1 = set("qwertyuiop")
    row2 = set("asdfghjkl")
    row3 = set("zxcvbnm")

    for word in words:
        # Lowercase character set of current word
        word_set = set(word.lower())

        # Check subset condition for each row
        if word_set.issubset(row1) or word_set.issubset(row2) or word_set.issubset(row3):
            print(word)

if __name__ == "__main__":
    solve()
```

	Input	Expected	Got	
✓	4 Hello Alaska Dad Peace	Alaska Dad	Alaska Dad	✓
✗	1 omk	No words		✗
✓	2 adsfd afd	adsfd afd	adsfd afd	✓

Your code must pass all tests to earn any marks. Try again.

Show differences

Incorrect

Marks for this submission: 0.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
import sys
import ast

def count_operations(nums: tuple) -> int:
    """
    Counts the number of operations to make the tuple empty.
    Operation 1: If first element is minimum, remove it.
    Operation 2: Else, rotate first element to the end.
    """
    current_tuple = nums
    remaining_set = set(nums)
    total_operations = 0

    while current_tuple:
        first = current_tuple[0]
        min_val = min(remaining_set)

        if first == min_val:
            # First element is the smallest -> remove it from tuple and set
            current_tuple = current_tuple[1:]
            remaining_set.remove(first)
        else:
            # First element is not smallest -> move to end
            current_tuple = current_tuple[1:] + (first,)

        total_operations += 1

    return total_operations

def main():
    # Read all input from standard input
    input_text = sys.stdin.read().strip()
    if not input_text:
        return
```

```
        return

    # Parse tuple input safely (handles inputs like "(3, 4, -1)" or "3 4 -1")
    try:
        if input_text.startswith("(") and input_text.endswith(")"):
            parsed_data = ast.literal_eval(input_text)
            if isinstance(parsed_data, int):
                nums = (parsed_data,)
            else:
                nums = tuple(parsed_data)
        else:
            nums = tuple(map(int, input_text.replace(',', ' ').split()))
    except Exception:
        nums = tuple(map(int, input_text.replace('(', '').replace(')', '').replace(',', ' ').split()))

    print(count_operations(nums))

if __name__ == "__main__":
    main()
```

	Test	Expected	Got	
✓	print(count_operations((3, 4, -1)))	5	5	✓
✓	print(count_operations((1, 2, 3, 4, 5)))	5	5	✓
✓	print(count_operations((5, 4, 3, 2, 1)))	15	15	✓
✓	print(count_operations((42,)))	1	1	✓
✓	print(count_operations((-2, 3, -5, 4, 1)))	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

Input: t = (5, 6, 5, 7, 7, 8), K = 13
Output: 2
Explanation:
Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
import sys

def count_distinct_pairs(t: tuple, k: int) -> int:
    seen = set()
    distinct_pairs = set()

    for num in t:
        complement = k - num
        if complement in seen:
            # Store the pair in sorted order (smaller, larger) to ensure uniqueness
            pair = (min(num, complement), max(num, complement))
            distinct_pairs.add(pair)
        seen.add(num)

    return len(distinct_pairs)

def main():
    input_text = sys.stdin.read().splitlines()
    if not input_text:
        return

    # Extract non-empty lines
    lines = [line.strip() for line in input_text if line.strip()]
    if not lines:
        return

    # Parse the tuple from the first line
    tuple_line = lines[0]
    nums = tuple(map(int, tuple_line.replace('(', '').replace(')', '').replace(',', ' ').split()))

    # Parse K from the second line (if present, default to 0 otherwise)
    k = int(lines[1]) if len(lines) > 1 else 0

    print(count_distinct_pairs(nums, k))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓
✓	1,2,1,2,5 3	1	1	✓
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # First token is array size N
    n = int(input_data[0])

    # The next N tokens are the array elements
    elements = input_data[1:n+1]

    # Preserve order of first appearance using dict.fromkeys() or set tracking
    seen = set()
    distinct_elements = []

    for item in elements:
        if item not in seen:
            seen.add(item)
            distinct_elements.append(item)

    # Print distinct elements space-separated on a single line
    print(*(distinct_elements))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all whitespace-separated tokens from stdin
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # First two tokens are the sizes n and m
    n = int(input_data[0])
    m = int(input_data[1])

    # Extract elements for array 1 and array 2
    arr1 = [int(x) for x in input_data[2 : 2 + n]]
    arr2 = [int(x) for x in input_data[2 + n : 2 + n + m]]

    # Convert to sets for O(1) membership lookup
    set1 = set(arr1)
    set2 = set(arr2)

    non_repeating = []

    # Collect elements in arr1 not present in arr2 (preserving order)
    for x in arr1:
        if x not in set2 and x not in non_repeating:
```

```
            non_repeating.append(x)

    # Collect elements in arr2 not present in arr1 (preserving order)
    for x in arr2:
        if x not in set1 and x not in non_repeating:
            non_repeating.append(x)

    # Output result
    if not non_repeating:
        print("NO SUCH ELEMENTS")
    else:
        print(*non_repeating)
        print(len(non_repeating))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
import sys

def find_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return num
        seen.add(num)
    return None

def main():
    # Read all input from standard input split by whitespace
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Parse numbers from input
    nums = [int(x) for x in input_data]

    duplicate = find_duplicate(nums)
    if duplicate is not None:
        print(duplicate)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	1 3 4 4 2	4	4	✓
✓	1 2 2 3 4 5 6 7	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the **`10`-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```python
import sys

def find_repeated_dna_sequences(s: str):
    seen = set()
    repeated = set()

    # Extract all 10-letter substrings using a sliding window
    for i in range(len(s) - 9):
        sub = s[i : i + 10]
        if sub in seen:
            repeated.add(sub)
        else:
            seen.add(sub)

    return list(repeated)

def main():
    # Read input string from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    s = input_data[0]
    result = find_repeated_dna_sequences(s)

    # Print each 10-letter repeated sequence on a new line
    for seq in result:
        print(seq)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCCCCCCCAAAAA	AAAAACCCCCCCCCCAAAAA	✓
✓	AAAAAAAAAAAAA	AAAAAAAAAA	AAAAAAAAAA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all lines from standard input
    lines = [line.strip() for line in sys.stdin.read().splitlines() if line.strip()]
    if not lines:
        return

    # Check if test case count is passed on line 1
    if lines[0].isdigit():
        lines = lines[1:]

    if len(lines) < 3:
        return

    # Parse sets from three lines of space-separated strings
    set1 = set(lines[0].split())
    set2 = set(lines[1].split())
    set3 = set(lines[2].split())

    # Check subset conditions using issubset() method or <= operator
    is_subset_set1 = set3.issubset(set1)
    is_subset_set2 = set3.issubset(set2)

    if is_subset_set1 and is_subset_set2:
```

```
        print("yes")
        print("set3 is subset of set1 and set2")
    else:
        print("No")

if __name__ == "__main__":
    main()
```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✓	2	mango orange banana orange grapes	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read raw text input from standard input
    raw_input = sys.stdin.read().splitlines()

    # Filter out completely empty lines
    lines = [line for line in raw_input if line.strip() != ""]
    if not lines:
        return

    # First non-empty line is the text
    text = lines[0]

    # Second line is brokenLetters if present, otherwise empty string
    brokenLetters = lines[1].strip() if len(lines) > 1 else ""

    # Convert broken letters to lower set
    broken_set = set(brokenLetters.lower())

    # Split text into words
    words = text.split()
    count = 0

    for word in words:
        # Check if any character in word matches broken keys (case-insensitive)
        word_set = set(word.lower())
        if not (word_set & broken_set):
            count += 1

    print(count)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	hello world ad	1	1	✓

	Input	Expected	Got	
✓	Welcome to REC e	1	1	✓
✓	Faculty Upskilling in Python Programming ak	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
import sys

def is_binary_string(s: str) -> str:
    # Convert string into a set of unique characters
    char_set = set(s)

    # Valid binary characters set
    binary_set = {'0', '1'}

    # Check if char_set is a subset of binary_set
    if char_set.issubset(binary_set):
        return "Yes"
    else:
        return "No"

def main():
    # Read raw line from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Extract first line (preserving spaces if present)
    s = input_data[0].strip('\r\n')

    print(is_binary_string(s))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.